



Subject: Operating System Lab 8

Submitted by: Motashma Noor

Department: BSCS (5)

SAP_ID: 55232

Assignment: 1

Riphaah International University
Islamabad Campus

TASK 1:

```
student@student-virtual-machine:~$ gcc 1.c
student@student-virtual-machine:~$ ./a.out
Child 1 - PID: 3503, Loop: 1
Child 1 - PID: 3503, Loop: 2
Parent - PID: 3502, Loop: 1
Child 2 - PID: 3504, Loop: 1
Child 2 - PID: 3504, Loop: 2
Child 2 - PID: 3504, Loop: 3
Child 2 - PID: 3504, Loop: 4
Child 2 - PID: 3504, Loop: 5
Child 1 - PID: 3503, Loop: 3
Child 2 - PID: 3504, Loop: 6
Child 2 - PID: 3504, Loop: 7
Child 1 - PID: 3503, Loop: 4
Child 2 - PID: 3504, Loop: 8
Child 1 - PID: 3503, Loop: 5
Child 2 - PID: 3504, Loop: 9
Child 1 - PID: 3503, Loop: 6
Child 2 - PID: 3504, Loop: 10
Child 1 - PID: 3503, Loop: 7
Child 2 - PID: 3504, Loop: 11
Child 1 - PID: 3503, Loop: 8
Child 2 - PID: 3504, Loop: 12
Child 1 - PID: 3503, Loop: 9
Child 2 - PID: 3504, Loop: 13
Child 1 - PID: 3503, Loop: 10
Child 2 - PID: 3504, Loop: 14

#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
#include <stdlib.h>
int main() {
    pid_t first_fork = fork();
    if (first_fork == 0) {
        for (int i = 1; i <= 20; i++) {
            printf("Child 1 - PID: %d, Loop: %d\n", getpid(), i);
        }
        exit(0);
    } else {
        pid_t second_fork = fork();

        if (second_fork == 0) {
            for (int i = 1; i <= 20; i++) {
                printf("Child 2 - PID: %d, Loop: %d\n", getpid(), i);
            }
            exit(0);
        } else {
            for (int i = 1; i <= 20; i++) {
                printf("Parent - PID: %d, Loop: %d\n", getpid(), i);
            }
            wait(NULL);
            wait(NULL);
        }
    }
    return 0;
}
```

Task 2:

```

#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <stdlib.h>

int main(void) {
    pid_t pid;
    int children = 3;

    for (int i = 1; i <= children; ++i) {
        pid = fork();
        if (pid < 0) {
            perror("fork");
            return 1;
        }
        for (int i = 0; i < 3; i++) {
            pid_t pid = fork();
            if (pid == 0) {
                printf("Child %d - PID: %d, PPID: %d\n", i+1, getpid(), getppid());
                exit(0);
            }
        }
    }
    for (int i = 0; i < 3; i++) {
        wait(NULL);
    }
    printf("Parent - PID: %d\n", getpid());
    return 0;
}

```

```

student@student-virtual-machine:~$ pico 2.c
student@student-virtual-machine:~$ gcc 2.c
student@student-virtual-machine:~$ ./a.out
Child 1 - PID: 3522, PPID: 3521
Child 2 - PID: 3523, PPID: 3521
Child 3 - PID: 3524, PPID: 3521
Parent - PID: 3521
student@student-virtual-machine:~$

```

TASK #3:

A **System Call** is a way for a program to interact with the operating System.

When a user program needs to perform operations that require hardware access or os level privileges such as reading a file, creating a process it cannot do these directly. Instead, it makes a system call to request the OS to perform the task on its behalf.

Why Do We Need System Calls?

System calls are needed for File operations like reading, writing, creating, or deleting files. Process control such as creating or terminating processes. Device management such as using a printer or scanner. Communication for sending or receiving data over a network. Memory management to allocate or free system memory.

Types of System Calls

System calls can be divided into several categories:

a) Process Control

Used to create and manage processes.

Examples: fork(), exec(), wait(), exit()

b) File Management

Used for creating, reading, writing, and closing files.

Examples: open(), read(), write(), close()

c) Device Management

Used to control and access hardware devices.

Examples: read(), write()

d) Information Maintenance

Used to get or set system data like process ID or user ID.

Examples: getpid(), getppid(), getuid()

e) Communication

Used for sending data between processes or across networks.

Examples: pipe(), send(),

Examples :

fork(): Creates a new process (child) that runs alongside the parent.

wait(): Parent pauses until the child process finishes.

exit(): Ends the process and cleans up memory.